Registro e inicio de sesión

Diseño y desarrollo de servicios web - caso

GA7-220501096-AA5-EV01

DATOS DE LA EVIDENCIA	
Aprendiz	Jhon Edwards Tineo Landaeta
Programa	Análisis y Desarrollo de Software
Ficha	3235874
Fecha	30 de octubre de 2026
Tecnologías	JavaScript, Node.js 24.x, SQLite, Git y GitHub

Introducción

Este proyecto implementa un servicio HTTP reutilizable para registrar usuarios y verificar sus credenciales. Recibe usuario y contraseña en formato JSON y entrega una respuesta clara de éxito o error. Incluye una pantalla de demostración que consume la misma API.

Objetivo

Diseñar, codificar y comprobar un servicio de registro e inicio de sesión con validaciones, persistencia, código comentado y control de versiones, de acuerdo con el caso planteado en la evidencia.

Alcance: la autenticación verifica credenciales y devuelve el mensaje solicitado. No genera un token ni mantiene una sesión persistente.

Contrato y organización

Operaciones HTTP

Método y ruta	Resultado esperado
POST /api/registro	201: registro satisfactorio. 409: usuario ya registrado.
POST /api/login	200: Autenticación satisfactoria. 401: Error en la autenticación.
GET /api/salud	200: servicio disponible.
GET /api/openapi.json	Contrato del servicio en OpenAPI 3.1.

Entrada común

```
Content-Type: application/json
{"usuario":"jhon.demo",
"contrasena":"Solar2026!"}
```

Organización por responsabilidades

Componente	Responsabilidad
app.js y server.js	Recibir HTTP, enrutar solicitudes, devolver JSON e iniciar/cerrar el servidor.
validation.js	Validar tipos, longitudes, caracteres permitidos y campos esperados.
auth-service.js	Aplicar reglas de registro e ingreso y construir respuestas públicas.
passwords.js	Crear y verificar hashes scrypt con sal individual.
user-repository.js	Crear la tabla y consultar SQLite con parámetros.
rate-limit.js	Controlar el número de solicitudes por IP.
public/ y tests/	Consumir la API desde formularios y comprobar su comportamiento.

Secuencia de cada operación

Registro: recibir datos, validar, comprobar duplicado, calcular hash, insertar cuenta y responder 201.
Ingreso: recibir datos, validar, consultar cuenta, comprobar hash y responder 200 o 401.

02 / VALIDACIONES

Datos y protección de credenciales

Regla	Aplicación
Usuario	Texto de 3 a 30 caracteres. Comienza con letra o número; admite letras ASCII, números, punto, guion y guion bajo. Se recortan extremos y se convierte a minúsculas.
Contraseña	Texto de 8 a 128 unidades UTF-16, no compuesto solo por espacios. No se recorta ni transforma. Se permiten caracteres especiales.
Formato	Objeto JSON con usuario y contrasena. Campos ausentes, tipos incorrectos, propiedades adicionales o JSON inválido: 400.
Duplicados	El nombre es único en SQLite; registros repetidos o simultáneos del mismo usuario: 409.
Tamaño y contenido	Más de 4096 bytes: 413. Un tipo de contenido distinto de application/json o contenido comprimido: 415.
Intentos	20 solicitudes de registro/ingreso por IP cada 10 minutos. Al superar el límite: 429 y cabecera Retry-After.

Modelo de datos: tabla usuarios

Campo	Tipo y restricción
id	INTEGER, clave primaria
usuario	TEXT, único y obligatorio
password_hash	TEXT, obligatorio; incluye algoritmo, parámetros, sal y hash
creado_en	TEXT, obligatorio; fecha/hora UTC ISO 8601

Decisiones de implementación

Se usa scrypt con N=16384, r=8, p=5, sal aleatoria de 16 bytes y clave derivada de 64 bytes. La comparación emplea timingSafeEqual. La API nunca devuelve la contraseña ni el hash. Para cuentas inexistentes también realiza una derivación y devuelve el mismo mensaje de error.

La base se crea en data/usuarios.sqlite y se conserva entre reinicios. El servidor escucha solo en 127.0.0.1. El limitador vive en memoria y se reinicia con el proceso; este alcance corresponde a la demostración local.

03 / EJECUCIÓN Y VERIFICACIÓN

Cómo abrir y comprobar la entrega

Paso a paso en Windows

1. Extrae todo el ZIP.
2. Instala Node.js 24.x desde nodejs.org/en/download.
3. Entra en la carpeta AA5-EV01 y abre INICIAR_WINDOWS.bat.
4. Mantén abierta la terminal y visita http://127.0.0.1:3000.
5. Crea una cuenta, inicia sesión y prueba una contraseña incorrecta.

Desde la terminal de VS Code

```
node --version
node src/server.js
```

Para ejecutar las pruebas, abre otra terminal en AA5-EV01:

```
node --test tests/api.test.js
```

Verificación	Resultado registrado
Pruebas automáticas	40 aprobadas, 0 fallidas. Ejecutadas por HTTP con Node.js v24.22.1 y bases temporales reales.
Cobertura funcional	Registro, acceso correcto e incorrecto, validaciones, duplicados simultáneos y persistencia al reiniciar.
Cobertura técnica	Hashes distintos con igual contraseña, límites de solicitudes, tamaño, formato JSON, métodos HTTP y cabeceras.
Reproducción	La salida está en docs/RESULTADOS_PRUEBAS.txt. Las pruebas no alteran las cuentas de demostración.

Repositorio e historial de Git

Repositorio público: github.com/Joan-dot650/Programa-Git.
Rama: evidencia-aa5-ev01. Carpeta: AA5-EV01. El enlace completo está en ENLACE_REPOSITORIO.txt. El ZIP incluye el historial en versionamiento/Programa-Git.bundle.

Archivos de apoyo y referencias

README.md: guía ampliada y solución de errores. docs/DISENO.md: diseño editable. docs/openapi.json: contrato. ejemplos/: peticiones HTTP y colección Postman.
Referencias: instrumento SENA IE-GA7-220501096-AA5-EV01; documentación de Node.js 24 (HTTP, SQLite, Crypto y Test runner); documentación de Git. Los enlaces se incluyen en DISENO.md.